

Value Stream Mapping

Enxergar o fluxo inteiro – da ideia ao deploy

Processos de Software — Sprint 3 (teoria + atividade)

Prof. Fernando · UFRN · 2026.1

De onde paramos

Sprint 2: o time aprendeu a **automatizar** (pipeline, Docker) e a **medir** (métricas DORA).

Já temos	Hoje
Pipeline rodando	Olhar o fluxo inteiro , não só o CI
Dois números DORA	Entender de onde vêm esses números
"A entrega está mais rápida"	Onde exatamente o tempo é gasto — e perdido

A Sprint 3 é a mais **reflexiva** do semestre: sai do "fazer" e entra no "analisar". Hoje começa pela ferramenta central — o VSM.

Plano da aula

1. O que é um Value Stream — e por que mapear ~10 min
2. Anatomia de um VSM: processos, filas, métricas ~15 min
3. Lead time vs cycle time vs %C&A ~10 min
4. **Atividade**: mapear o estado atual do seu projeto ~25 min

O que é um Value Stream

Um **fluxo de valor** é toda a sequência de passos para transformar uma **ideia** em **valor entregue** ao usuário.

IDEIA → backlog → análise → código → review → teste → deploy → USUÁRIO

A pergunta do VSM não é "esse passo é rápido?". É: **"da ideia ao usuário, quanto tempo passa — e quanto desse tempo foi realmente trabalho?"**

VSM vem do Lean da Toyota (lembra da Sprint 1: *muda, mura, muri*). Aqui aplicamos a mesma lente ao processo de software.

Por que mapear o fluxo todo

A intuição engana. Times "sentem" onde está o problema — e erram.

Percepção do time: "código é a parte lenta – somos poucos devs"
Realidade do VSM: código leva 4h; mas o PR fica 3 DIAS esperando review

Sem VSM	Com VSM
Otimiza o que é visível (escrever código)	Otimiza o que é lento (a fila do review)
Decisão por opinião	Decisão por dado
Melhoria local, ganho global zero	Ataca o gargalo de verdade

Otimizar um passo que não é o gargalo **não acelera nada**. O VSM existe para você não gastar esforço no lugar errado.

Trabalho vs espera

A revelação central do VSM: a maior parte do lead time costuma ser **espera**, não trabalho.

```
| análise | ESPERA | código | ESPERA | review | ESPERA | deploy |  
  2h      1 dia    4h      3 dias    1h      4h      20min  
└─────────┴─────────┴─────────┴─────────┴─────────┴─────────┴─────────┘  
trabalho: ~7h  
└─────────┴─────────┴─────────┴─────────┴─────────┴─────────┴─────────┘  
lead time total: ~4,7 dias
```

Tipo de tempo	No exemplo	É...
Trabalho (agrega valor)	~7h	o que o cliente "pagaria"
Espera / fila (não agrega)	~4 dias	<i>muda</i> — desperdício puro

Se ~85% do tempo é fila, contratar mais devs (acelerar as ~7h de trabalho) quase não move o lead time. **Atacar as filas, sim.**

Anatomia de um VSM

Três elementos no diagrama:

Elemento	Símbolo	O que representa
Caixa de processo	retângulo	Um passo onde há trabalho (análise, código, review)
Fila / espera	triângulo	Item parado esperando o próximo passo
Linha do tempo	escada embaixo	Alterna tempo de trabalho e tempo de espera



Cada caixa carrega suas métricas. O triângulo entre as caixas é onde a maioria das equipes descobre que está perdendo seu lead time.

As três métricas de cada caixa

Métrica	Sigla	O que mede
Lead Time	LT	Tempo total: entrou na etapa → saiu (inclui espera)
Process Time	PT	Tempo de trabalho efetivo na etapa
% Completo e Acurado	%C&A	% de itens que chegam sem precisar retrabalho

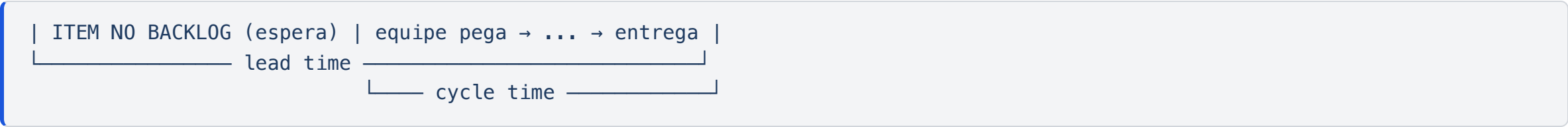
Etapa "Review": LT = 3 dias 1h | PT = 1h | %C&A = 70%

%C&A é a métrica que o pessoal esquece. Se a etapa de review recebe 30% dos PRs com problema (código que nem compila, sem teste), esse retrabalho **volta** para etapas anteriores — e infla o lead time de todo mundo.

Lead time vs cycle time

Dois termos que aparecem juntos e se confundem — voltam da Sprint 1 (Kanban):

Termo	Mede	Relógio
Lead time	Pedido feito → entregue	Começa quando o cliente pede
Cycle time	Trabalho começou → terminou	Começa quando a equipe pega



A diferença entre os dois é **tempo na fila do backlog**. Cliente sente o lead time. Se o cycle time é bom mas o lead time é péssimo, o gargalo está **antes** do trabalho começar — na priorização.

A eficiência do fluxo

Uma razão simples resume a saúde do fluxo:

$$\text{Eficiência de fluxo} = \frac{\text{tempo de trabalho } (\Sigma \text{ PT})}{\text{lead time total}}$$

Exemplo: 7h de trabalho / 4,7 dias úteis (~37,6h) $\approx$ 18%

Eficiência típica	Leitura
5–15%	Comum — muito tempo em fila
25–40%	Fluxo bem cuidado
> 40%	Raro — fluxo muito enxuto

VSM e DORA: como se conectam

Não são ferramentas concorrentes — uma alimenta a outra:

DORA	VSM
Mede o resultado (lead time, frequência)	Explica o resultado (onde o tempo vai)
"Lead time = 4,7 dias"	"...porque review fica 3 dias na fila"
Diz que há um problema	Diz onde está o problema

Na Sprint 2 vocês coletaram o **número**. O VSM da Sprint 3 abre esse número e mostra **a anatomia** dele. Juntos: DORA aponta o sintoma, VSM localiza a causa.

Atividade: mapear o estado atual

Em equipe, agora — desenhem o VSM do **seu projeto** (papel, quadro ou Miro):

1. Listem as **etapas** do fluxo: ideia → backlog → código → review → CI → deploy.
2. Para cada etapa, estimem **PT** (trabalho) e **LT** (com espera).
3. Marquem as **filas** (triângulos) entre as etapas.
4. Calculem o **lead time total** e a **eficiência de fluxo**.
5. Anotem: onde está a **maior fila**?

Use dados **reais** do projeto — histórico de PRs no GitHub, datas de commit e merge (igual à coleta DORA da Sprint 2).

Onde buscar os dados reais

Vocês já coletaram material parecido na Sprint 2. Reaproveitem:

```
# tempo entre primeiro commit de uma branch e o merge na main
git log --pretty="%h %ci %s" main

# PRs e suas datas (data de abertura → data de merge = fila do review)
# → aba "Pull requests" → filtro "is:merged"
```

Etapa do VSM	Onde achar o tempo
Backlog → início	Data do card no GitHub Projects → 1º commit
Código	1º commit → PR aberto
Review (fila!)	PR aberto → PR aprovado
CI	Duração do workflow na aba Actions
Deploy	Merge → publicação

Não precisa de precisão de cronômetro. Ordem de grandeza ("horas" vs "dias") já revela o gargalo.

Erros comuns ao montar o VSM

Erro	Por que atrapalha
Mapear o fluxo "ideal" em vez do real	Esconde justamente o que você quer achar
Esquecer as filas (só caixas de processo)	A fila é onde mora o desperdício
Misturar PT e LT	Perde a distinção trabalho vs espera
Mapear granular demais (15 etapas)	Vira diagrama, não vira análise
Ignorar %C&A	Não enxerga o retrabalho escondido
Fazer sozinho, sem o time	Cada um conhece um pedaço do fluxo

Checklist

- ☐ Etapas do fluxo do projeto listadas (5–7 caixas)
- ☐ Filas marcadas entre as etapas
- ☐ PT e LT estimados por etapa, com dados reais
- ☐ Lead time total e eficiência de fluxo calculados
- ☐ Maior fila identificada — candidata a gargalo

Referências da aula

- Martin & Osterling — *Value Stream Mapping* (cap. 1) — leitura **L12**
- Rother & Shook — *Learning to See* — a obra clássica de VSM
- DORA — *Four Keys* — dora.dev/guides/dora-metrics-four-keys
- Kim et al. — *The DevOps Handbook* — fluxo de valor e o Primeiro Caminho
- Reinertsen — *The Principles of Product Development Flow* — teoria de filas